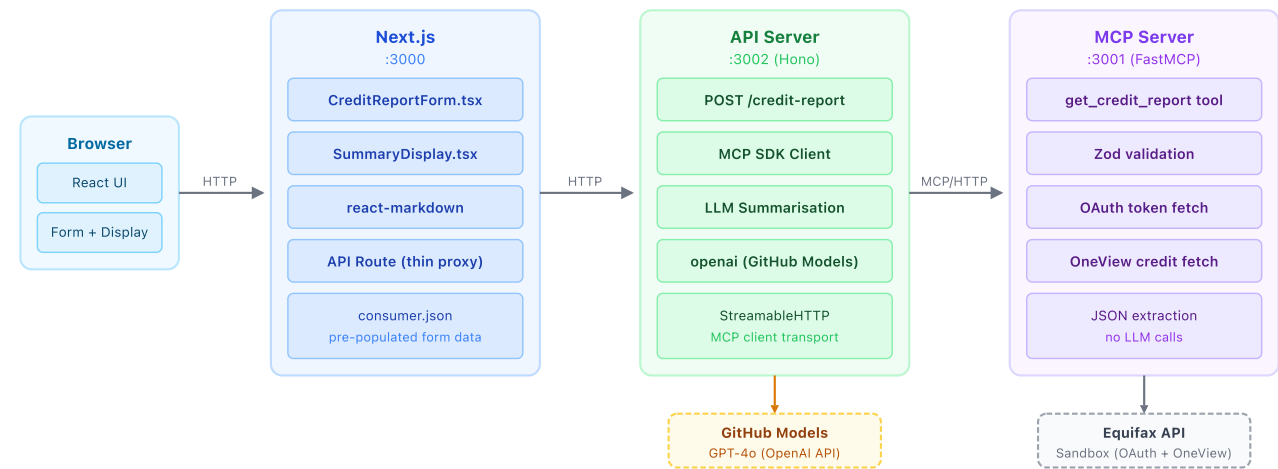


MCP Credit Report Demo — Architecture

Three-process TypeScript application: Next.js UI · Hono API Server · FastMCP Data Server

System Overview



Architecture Summary

The application is split into three independent processes that communicate over HTTP. The UI layer delegates all business logic to the API server, which in turn delegates all data fetching to the MCP server. No process holds another's credentials or concerns.

Port 3000

Next.js — Presentation

Serves the React UI. Reads `consumer.json` to pre-populate the form. The API route is a **thin proxy** — it forwards requests to the API server verbatim and returns the response with no business logic.

Port 3002

API Server — Business Logic

Built with **Hono** on Node.js. Acts as an MCP client, calls `get_credit_report` on the MCP server, then sends the structured result to **GPT-4o** via GitHub Models for a plain-English summary.

Port 3001

MCP Server — Data Layer

Built with **FastMCP**. Exposes a single tool that validates input with Zod, fetches an Equifax OAuth token, posts the OneView credit request, extracts key fields, and returns raw JSON. No LLM calls.

Separation of Concerns

UI, business logic, and data fetching live in separate processes. Each can be restarted, scaled, or replaced independently.

Input Validation

Consumer fields are validated with **Zod** at the MCP server boundary — the only point where external data enters the system.

MCP Protocol

The API server connects to the MCP server via `StreamableHTTPClientTransport`, enabling any MCP-compatible client to consume the credit report tool.

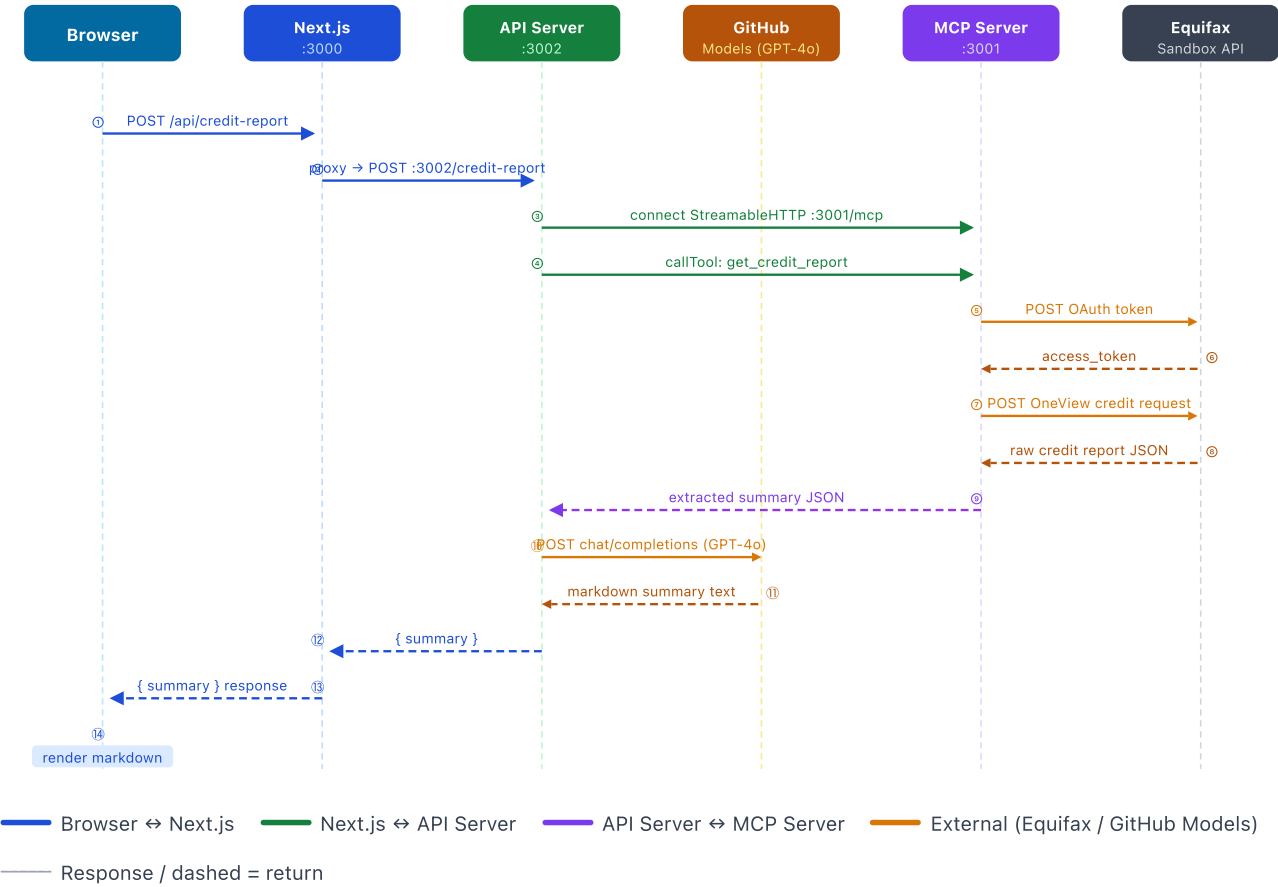
Credential Isolation

Equifax credentials live only in `mcp-server/.env`. The GitHub token lives only in `api-server/.env`. The Next.js process holds no secrets.

Request Flow

Color-coded by service channel — 14 steps from form submit to rendered summary

Sequence Diagram



Step Overview

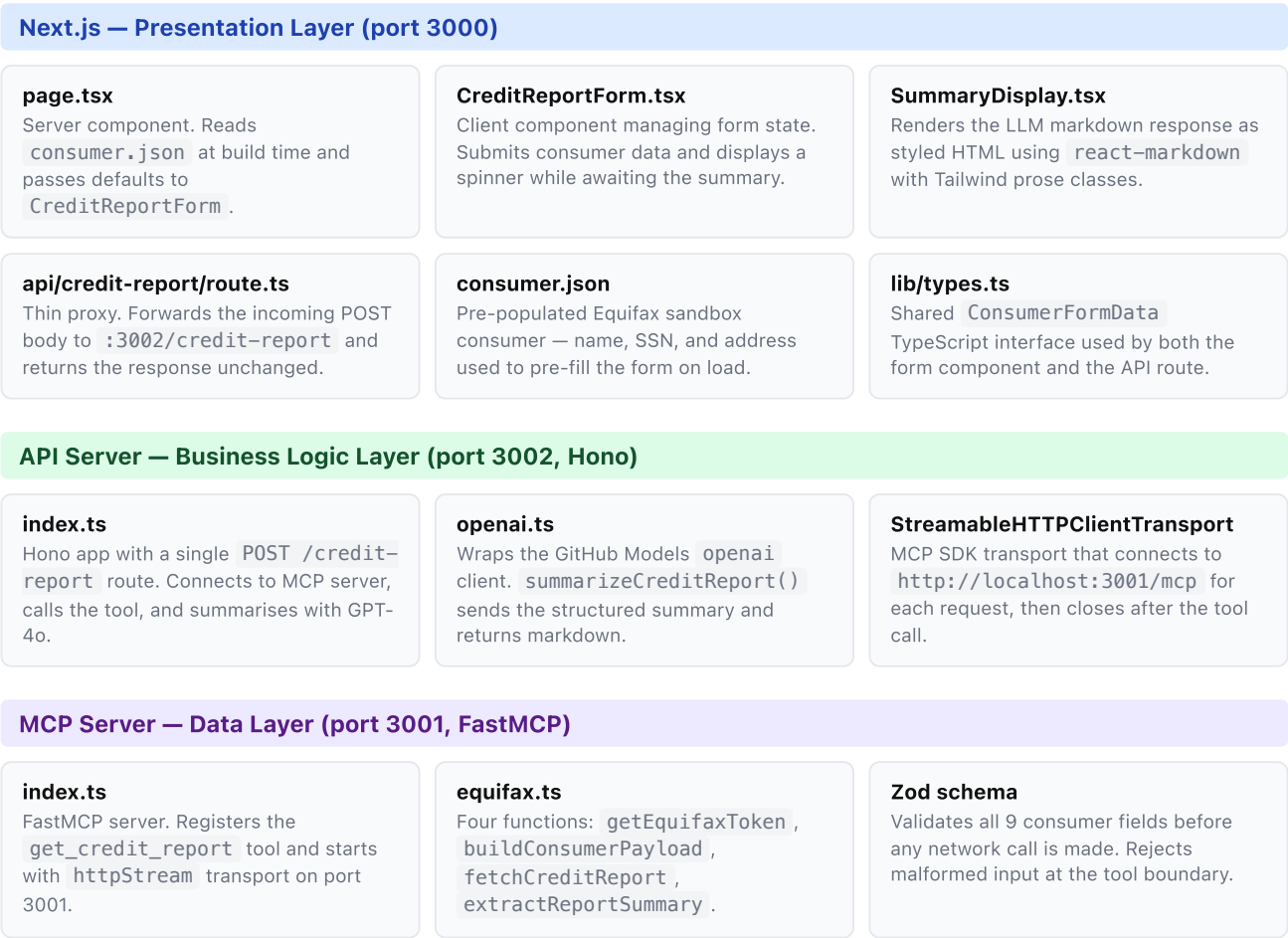
All 14 steps — color-coded by service channel

Step-by-Step Breakdown

1 User submits form Browser POSTs <code>ConsumerFormData</code> to Next.js <code>/api/credit-report</code>	2 Next.js proxies request API route forwards body to API server <code>POST :3002/credit-report</code>
3 MCP connection API server connects to MCP server via <code>StreamableHTTPClientTransport</code>	4 Tool invocation API server calls <code>get_credit_report</code> tool with consumer fields
5 OAuth token request MCP server POSTs client credentials to Equifax token endpoint	6 Token response Equifax returns <code>access_token</code> to MCP server
7 Credit report request MCP server POSTs OneView consumer credit request with bearer token	8 Credit report response Equifax returns raw credit report JSON to MCP server
9 Extraction & return MCP server extracts key fields and returns structured summary JSON to API server	10 LLM request API server POSTs summary JSON to GPT-4o via GitHub Models
11 LLM response GitHub Models returns plain-English markdown summary	12 API server responds Returns <code>{ summary }</code> JSON to Next.js
13 Next.js responds Forwards <code>{ summary }</code> response to browser	14 Browser renders <code>SummaryDisplay.tsx</code> renders markdown as formatted HTML via <code>react-markdown</code>

Component Detail

Key files and their responsibilities within each process



Key Dependencies

npm packages by process and purpose

Package Reference

Package	Process	Purpose
next , react , react-dom	Next.js	React framework and runtime
react-markdown	Next.js	Render LLM markdown output as HTML
tailwindcss	Next.js	Utility-first CSS for styling
typescript	All	Static typing across all three processes
hono , @hono/node-server	API Server	Lightweight HTTP framework on Node.js
@modelcontextprotocol/sdk	API Server	MCP client — StreamableHTTPClientTransport
openai	API Server	GitHub Models (GPT-4o) via OpenAI-compatible API
fastmcp	MCP Server	MCP server framework with tool registration and HTTP transport
zod	MCP Server	Runtime validation of tool input parameters
dotenv	API + MCP Server	Load .env secrets into process.env
tsx	API + MCP Server	Run TypeScript directly with npm run dev

Environment Variables

Variable	File	Used by
EQUIFAX_USERNAME	mcp-server/.env	MCP Server — Equifax OAuth client ID
EQUIFAX_PASSWORD	mcp-server/.env	MCP Server — Equifax OAuth client secret
GITHUB_TOKEN	api-server/.env	API Server — GitHub Models bearer token

Running the Application

Terminal	Command	Service
1	cd mcp-server && npm run dev	MCP Server on port 3001
2	cd api-server && npm run dev	API Server on port 3002
3	npm run dev (project root)	Next.js on port 3000